Storyline

Integration Tests

Generate good system test cases from existing catalog of scenario types

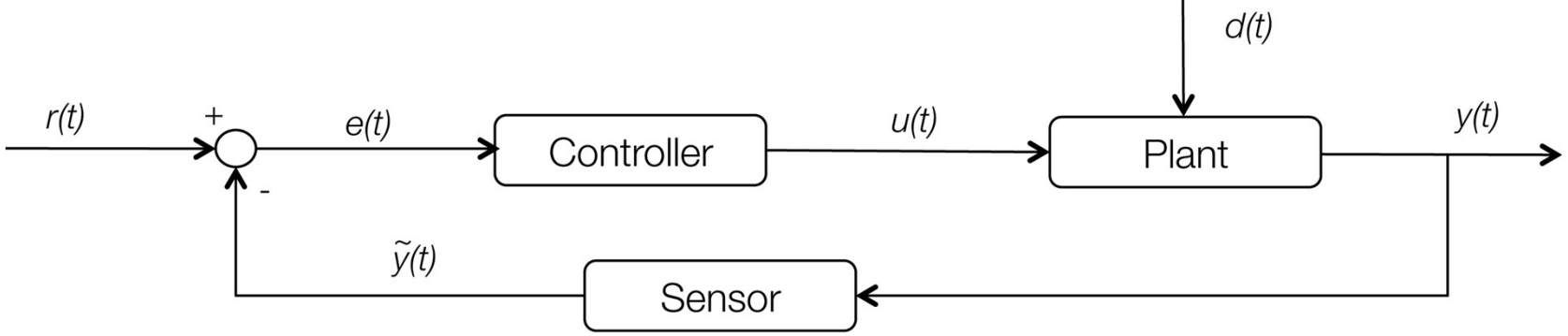
- **No regression tests for closed-loop systems!**
- **Optimization problem: describe scenario types and objective functions**
- **Fitness functions**
- **Revisiting Re-Use**
- **STL**
- **Completeness**

Derive parameterized scenarios from data

- Clustering
- Clusters and their semantics
- Completeness revisited: recorded drives

Discussion and Conclusions

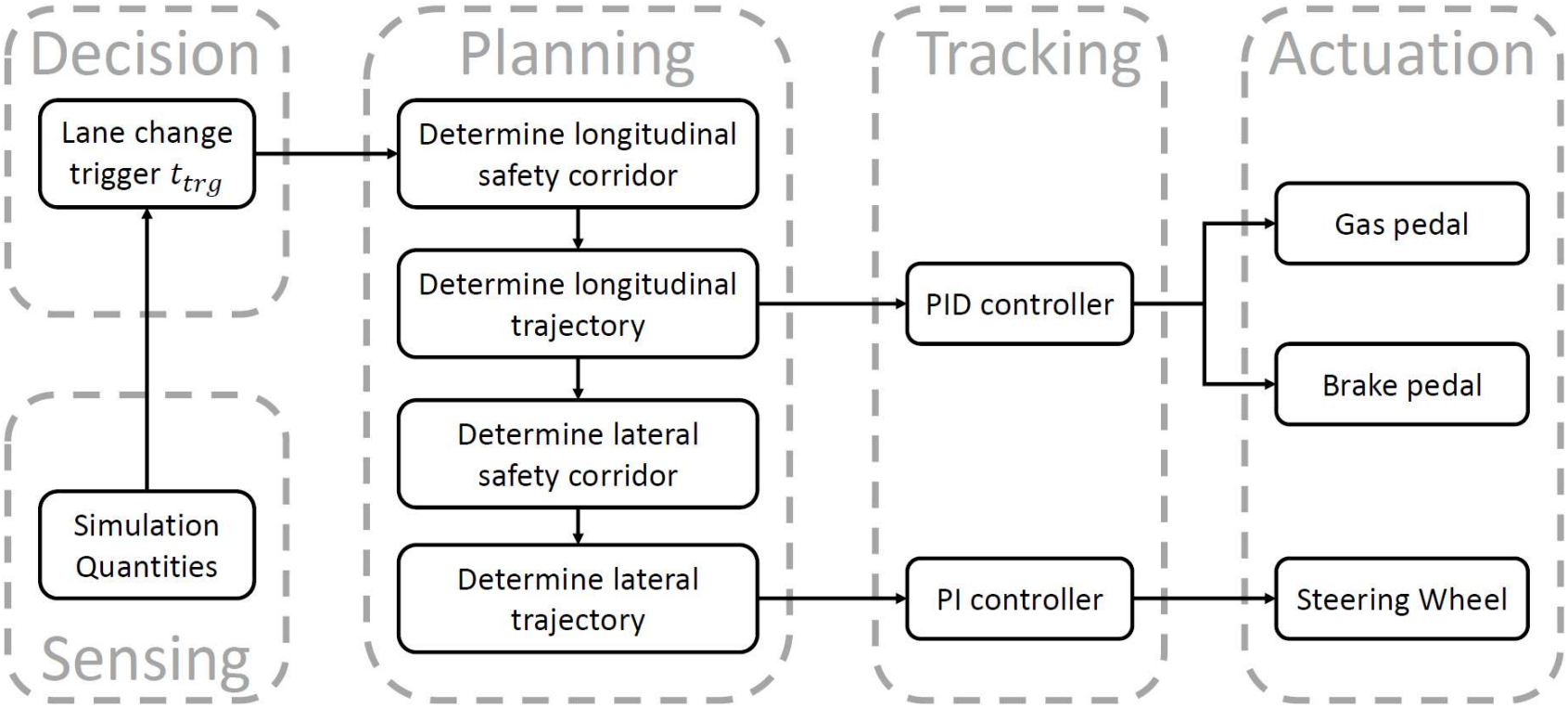
Continuous/Hybrid Closed-Loop, not Discrete



- Controller receives difference of desired value r(t) and sensor measurement $\tilde{y}(t)$, i.e. error e(t)
  - Example air conditioning: desired and measured temperatures
- Controller computes and forwards actuator control value u(t)
  - Example AC: degree of opening/closing of valve that controls cooling liquid
- Plant maps u(t) to actual y(t) by considering environmental disturbance d(t)
  - Example UC: cooling liquid has effect on actual room temperature
- Sensor observes y(t) and forward the measurement $\tilde{y}(t)$

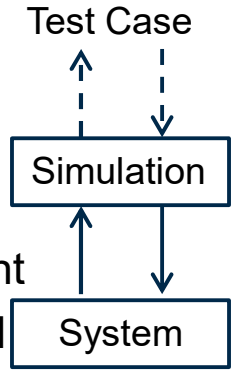
[Holling, Pretschner (2016), Failure Models for Testing Continuous Controllers]

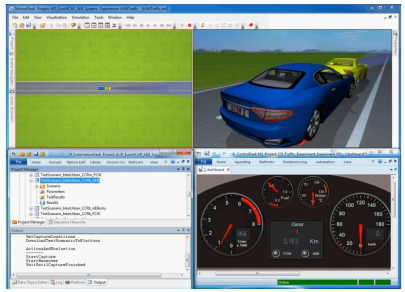
Example System



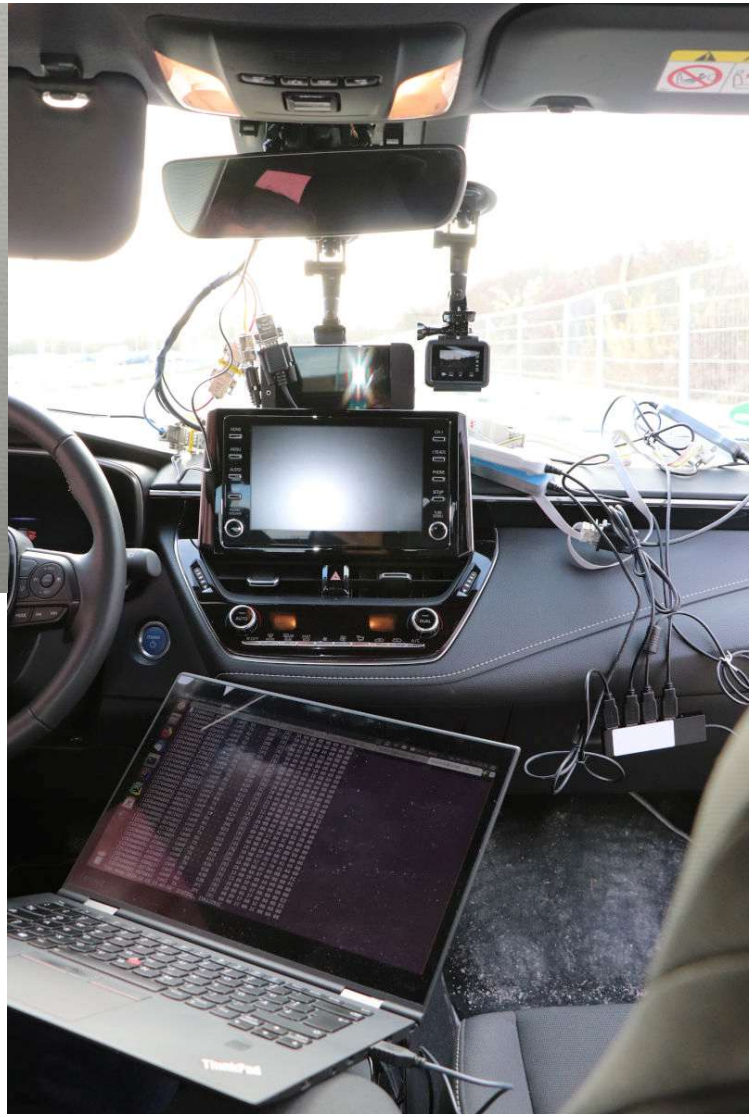
X-in-the-Loop Testing

- Controllers usually parts of a bigger (discrete-continuous) system
- Test setups vary depending on the development status
- X-in-the-Loop test setups rely on a simulation environment
  - Model-in-the-Loop: Model of the system is run in the simulation environment
  - Software-in-the-Loop: Actual software is used; sometimes on the dedicated computation hardware
  - Hardware-in-the-Loop: Actual hardware is used, e.g. mechanical steering parts
  - (Vehicle-in-the-Loop for autonomous driving systems: The whole car is used)
- Goal: gradually improve "realism" of the test setup





Software-in-the-Loop, Vehicle-in-the-Loop



Testing the Untestable

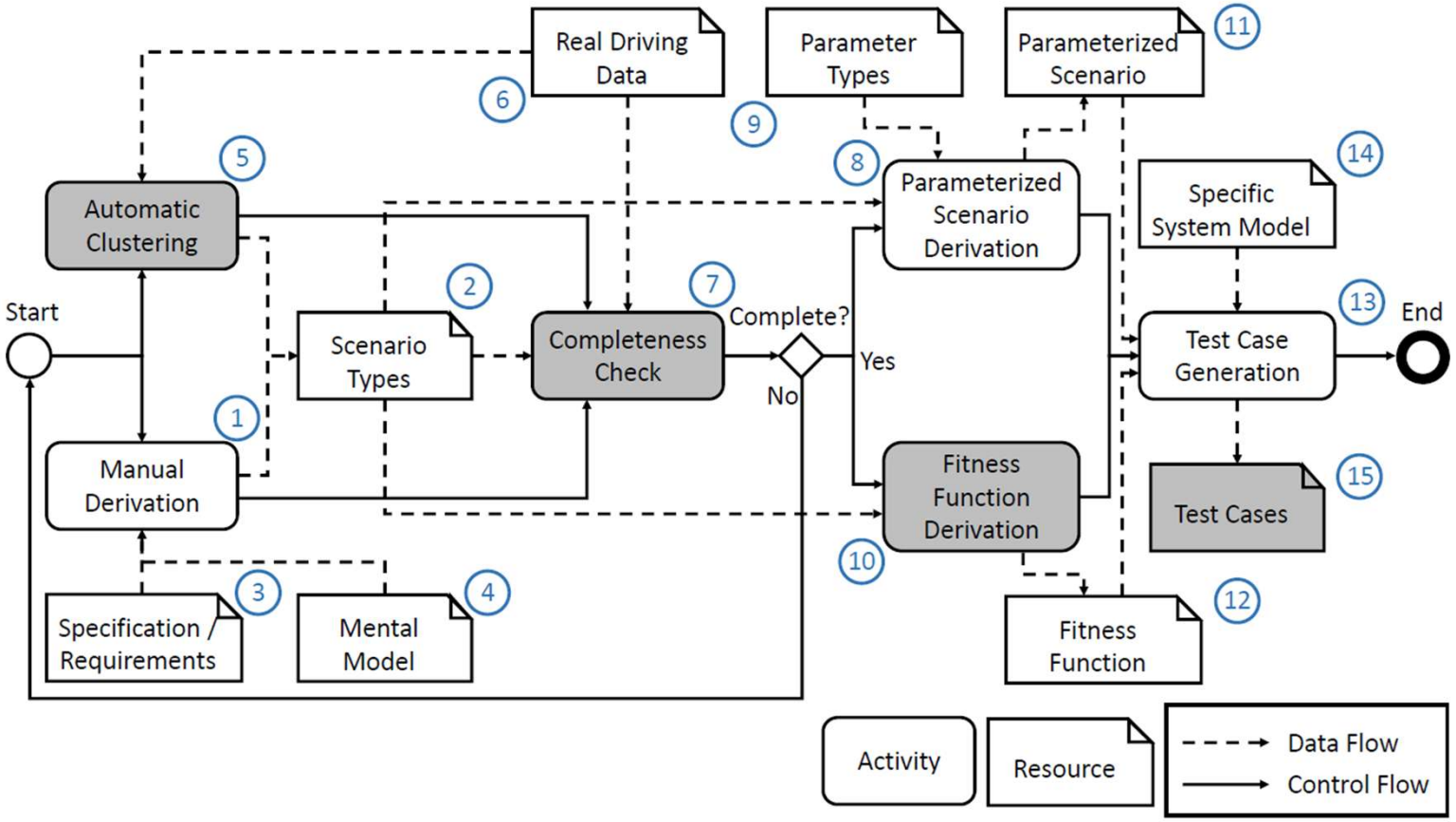
What is the specification of an autonomous vehicle?

By methodology, no specifications for ML-based components

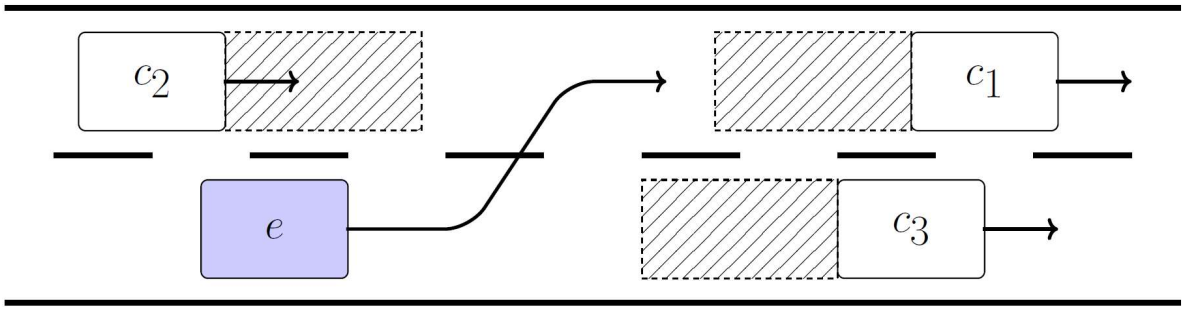
Need to reverse-engineer specification

- (Mis)behavior in traffic scenarios
- „no crash“
- „follow informal traffic rules“

Overview



No Re-Use of Recorded Drives for testing ADAS!



Test whether ego vehicle *e* keeps sufficient distance to surrounding cars during lane change

- System version e_1 violates distance to c_1
- System is “corrected”
- System version e_2 does not even perform a lane change

One test case can be **good** for one system and **bad** for another! **Re-Use is random!**

[Hauer, P, Holzmüller : Re-Using Concrete Test Scenarios Generally Is a Bad Idea, IEEE IV 2020]

Remember: Good Tests

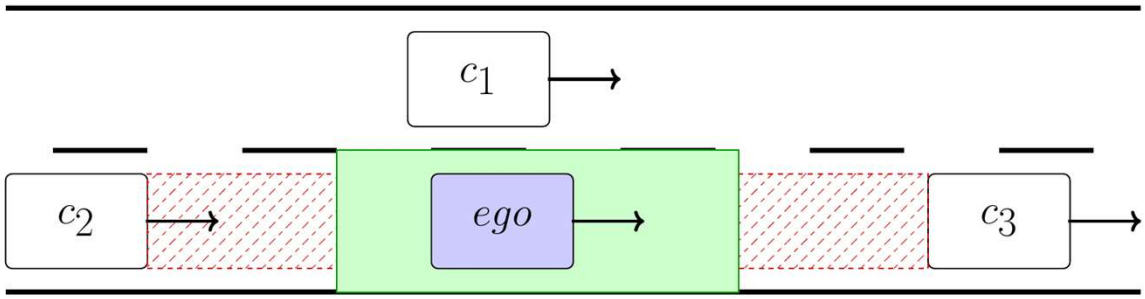
Test: Input plus expected output (plus infrastructure conditions) → require a specification!

~~„reveal faults“~~

~~„reveal *potential* faults“~~

„reveal potential bugs with good cost effectiveness (low risk)“

“Good” Test Cases



Safe Operating Envelope

aligned to: Koopman, Challenges in Autonomous Vehicle Testing and Validation, 2016

A “good” test case reveals potential *faulty system behavior with good cost effectiveness.*

aligned to: Pretschner, Defect-Based Testing, 2015

In a “good” test scenario, a

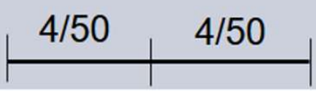
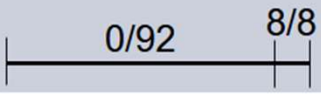
- correct system approaches
- faulty system violates

the limits of the safe operating envelope.

Speaking of … Random Tests?

Testing usually based on partitioning the input domain, then sampling from each block
Depending on distribution of failing inputs, random testing performs better/worse/same!

For instance, 2 blocks and 1 test from each block for „probability to reveal at least one failure“

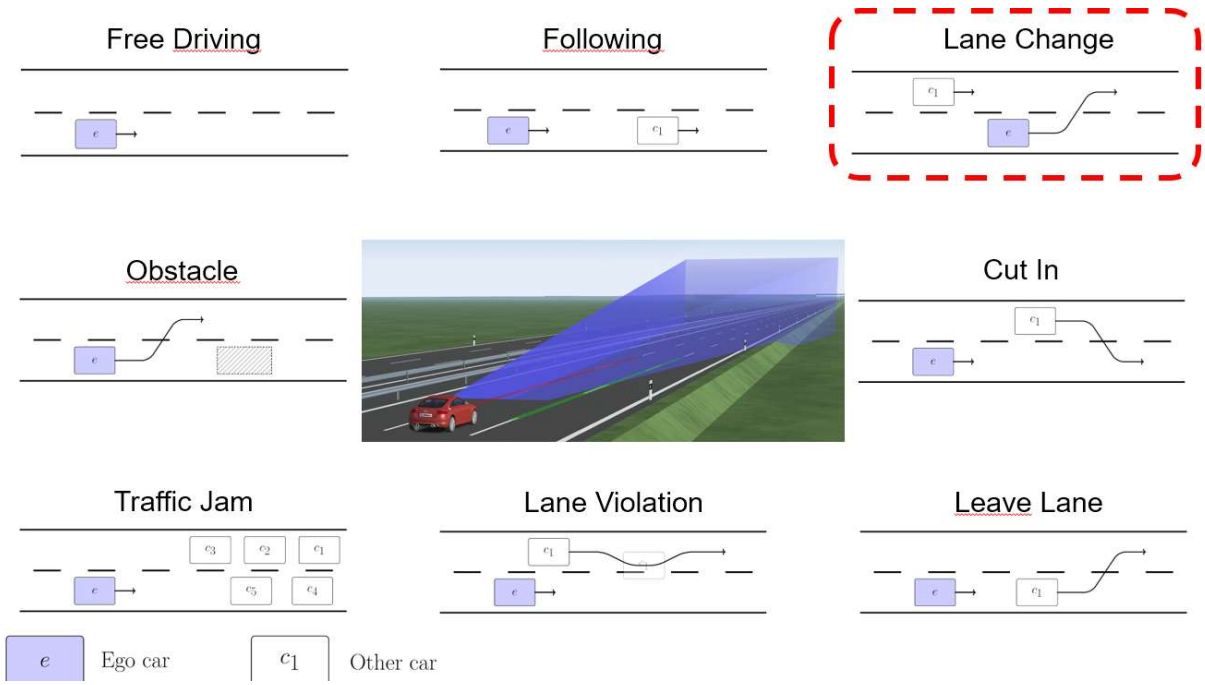
No difference	Partition testing wins	Random testing wins
N=2	N=2	N=2
Both blocks contain same number of failure-causing inputs	All and only failure causing inputs in one block	Some (here one) small blocks contain only correct inputs
		

Hence need to choose blocks in a risk-based manner (defect hypothesis, harm, likelihood)

CPS: Huge input spaces and few test cases

Scenario-Based Testing for ADAS

Scenarios as blocks of an input space partition not defect-based but occur frequently (risk!)
Extreme scenario instances are limit tests, hence defect-based
Traceability/interpretability required by regulation



Partition-Based Testing?

The small images on the slide before represent *scenario types*: They provide an intuitive understanding of a given traffic situation and the target behavior of the autonomous car (or the respective controller).

Env: $\mathbb{R}_+ \rightarrow (\mathbb{R}_+ \times \mathbb{R}_+)^n$ describes the environment: the geometric (x,y) position at each moment in time for each of the *n* non-player characters (vehicles, humans in the environment); and

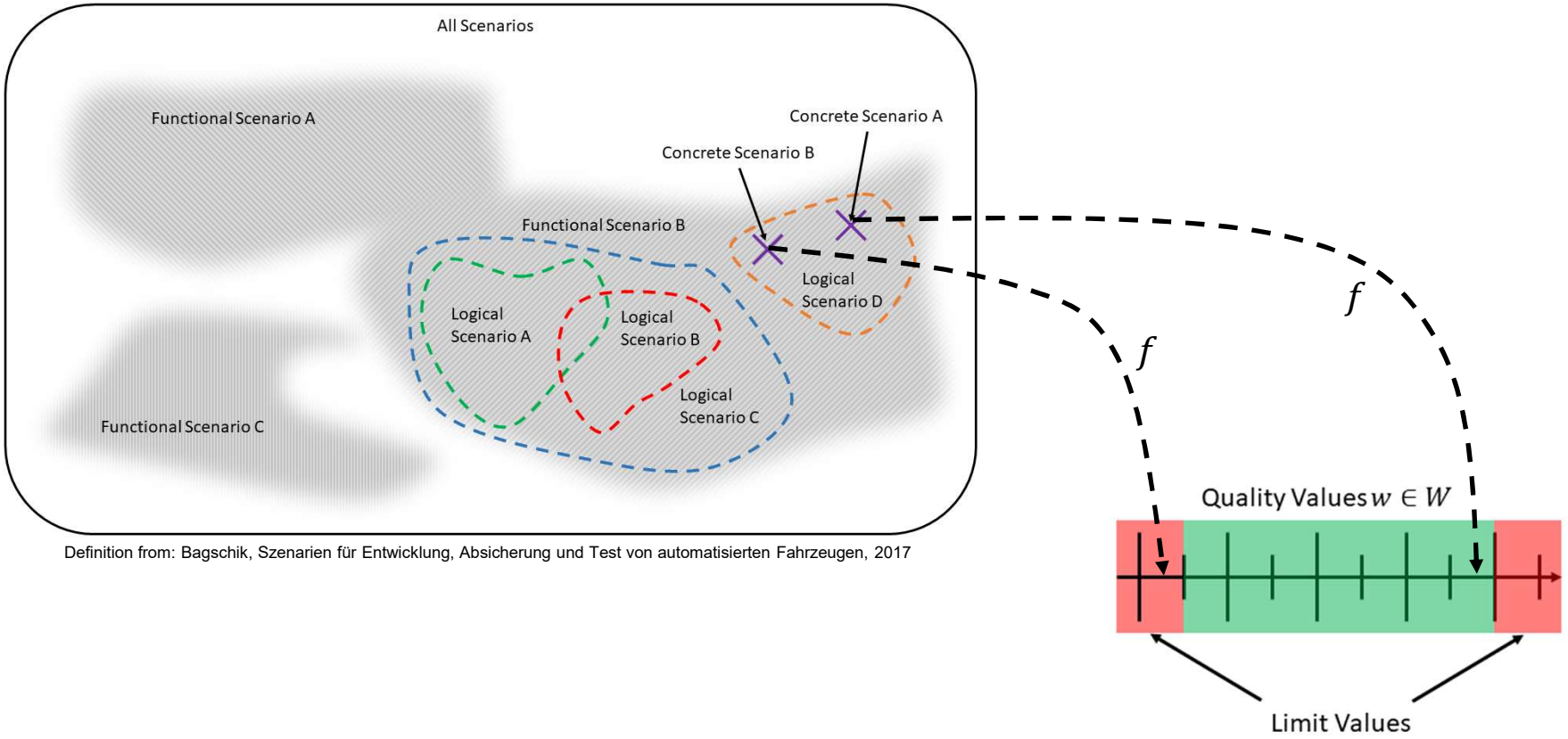
Bhv: $\mathbb{R}_+ \rightarrow \mathbb{R}_+ \times \mathbb{R}_+$ describes the ego vehicle‘s behavior: the geometric (x,y) position at each moment in time.

A given functionality such as „lane following“ or „overtake when car is approaching from behind“ is defined for only some possible environments, called the *operational design domain* $ODD \subseteq Env$;
and gives rise to a desired target behavior of the ego vehicle $Trg \subseteq Bhv$.

A *logical scenario* (or *specification scenario*) then is the specification of a function $ODD \rightarrow Trg$, which includes the ranges of all possible parameters in the ODD. It reflects the intuition of its respective scenario type.

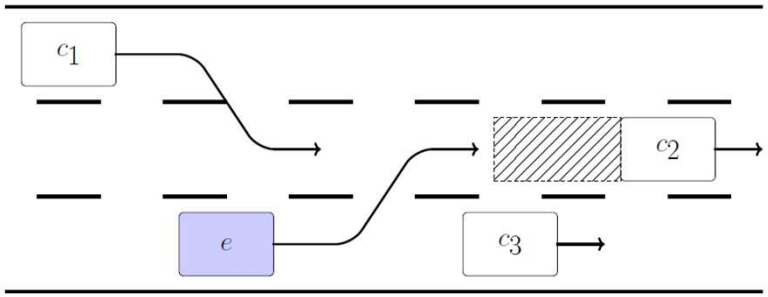
(Our notion of) *scenario-based testing* then means to (1) partition Env into multiple ODDs; and (2) subsequently partition the ODD of a given ADAS functionality and search for those limit values in each block that potentially create crashes, or come close to a crash.

Scenario-Based Testing and Limit Testing



Definition from: Bagschik, Szenarien für Entwicklung, Absicherung und Test von automatisierten Fahrzeugen, 2017

Logical Scenarios, Limit Tests, Optimization



Parameter	Lower Bound	Upper Bound
Starting positions s_{0,c_i} [m]	0	300
Starting times t_{start,c_i} [s]	0	3
Target velocities v_i [km/h]	80	140
Lane change time t_{lc,c_1} [s]	6	13

Simplified 10-dimensional space of test cases. But not all candidates

- in the space are test cases of the desired form
- of the desired form are “good” test cases

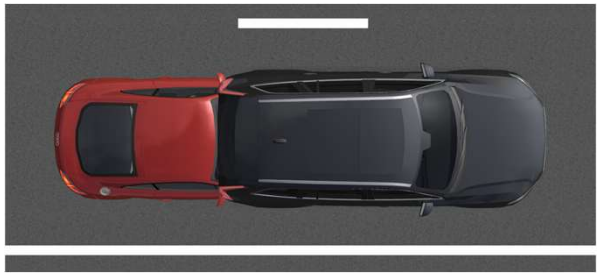
Formulate as optimization problem: (“Good”=“Optimal”!)

- lane change of e behind c_2 && simultaneous lane change of c_1 && lane change of c_1 behind e
- AND search for envelope violation

Our contribution: Methodology, fitness function templates

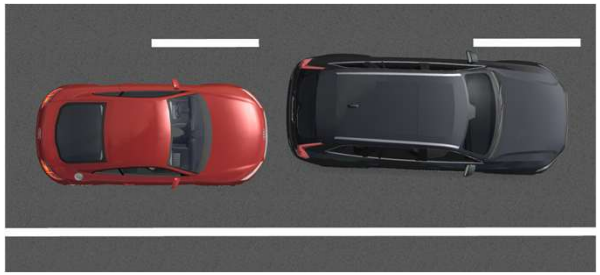
[Hauer, P, Holzmüller: Fitness Functions for Testing Automated and Autonomous Driving Systems. SAFECOMP 2019]

Example: Unsafe System



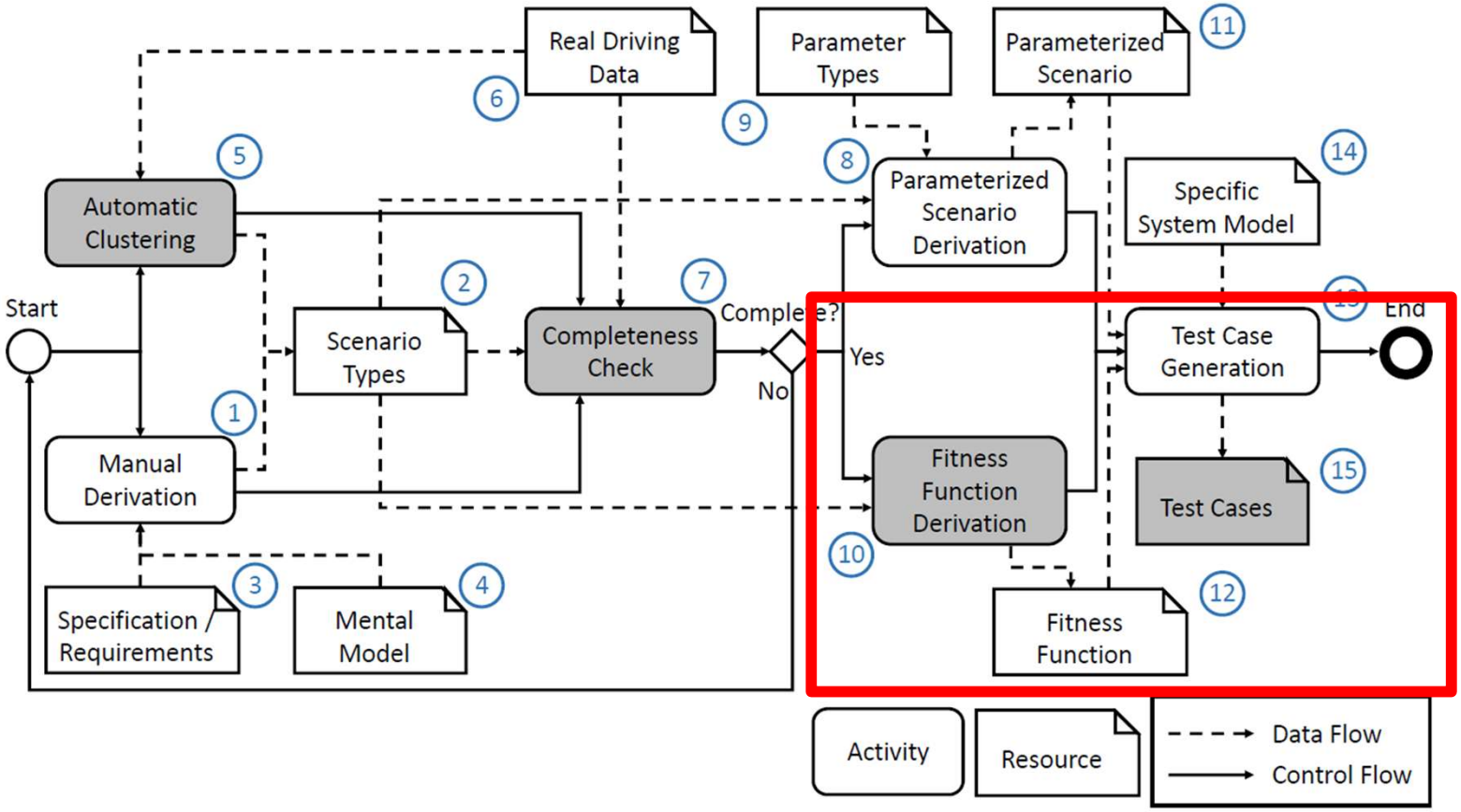
- Ego car is responsible for crash: didn‘t break sufficiently fast
- Collision: violation of the allowed braking distance by 3.49m

Example: Safe System



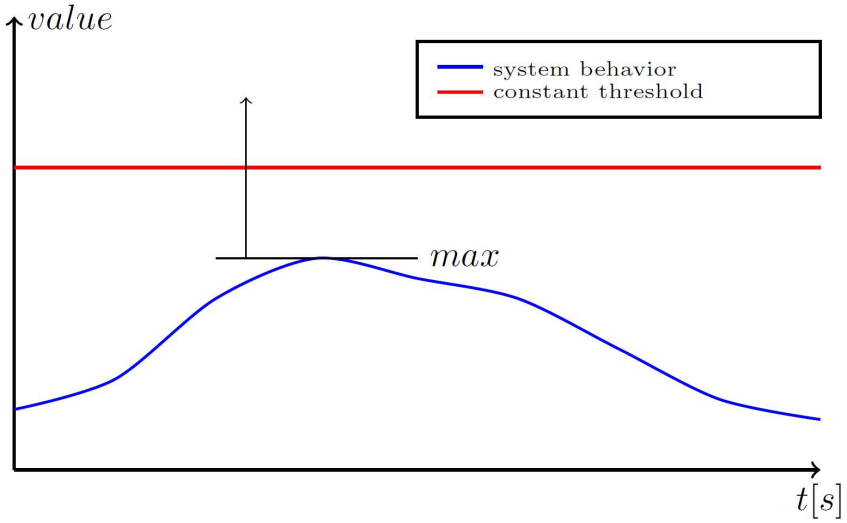
- Ego car did break sufficiently fast
- No collision; 0.37m less braking distance needed than allowed

Fitness Functions



Fitness Functions: The Foundation

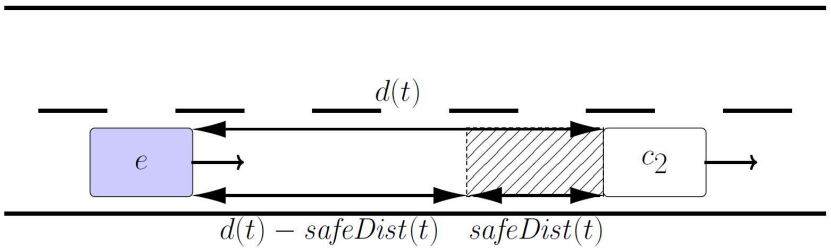
“System behavior must not exceed a safety threshold.”

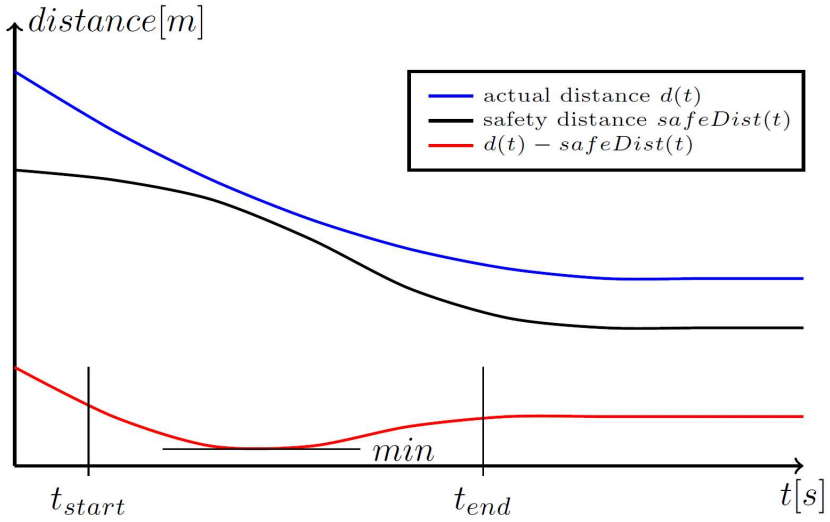


$$f_{\text{idea},1} = -\max(blue\ curve)$$

Template 1 – Testing the Safe Operating Envelope

“Ego must not leave the safe operating envelope”
Search goal: Violate the safe operating envelope





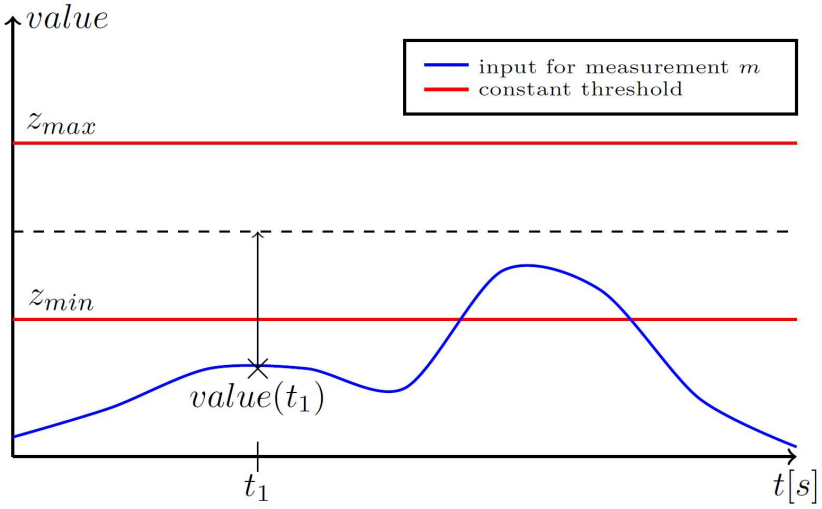
$$f = \min\{d(t) - safeDist(t)\} \qquad t \in [t_{start}, t_{end}]$$

$$f = \min\{actualValue(t) - envelopeTrsh(t)\} \qquad t \in [t_{start}, t_{end}]$$

Qualitative Test Goals

"$value(t_1)$ has to be within a certain range."

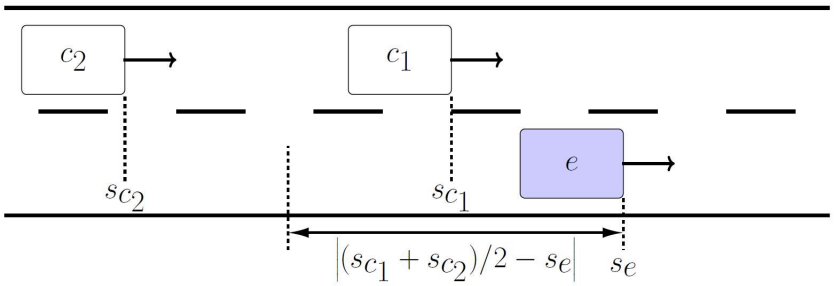
$$f_{\text{idea},2} = \begin{cases} m, & \text{qualitative test goal not fulfilled} \\ 0, & \text{otherwise} \end{cases}$$

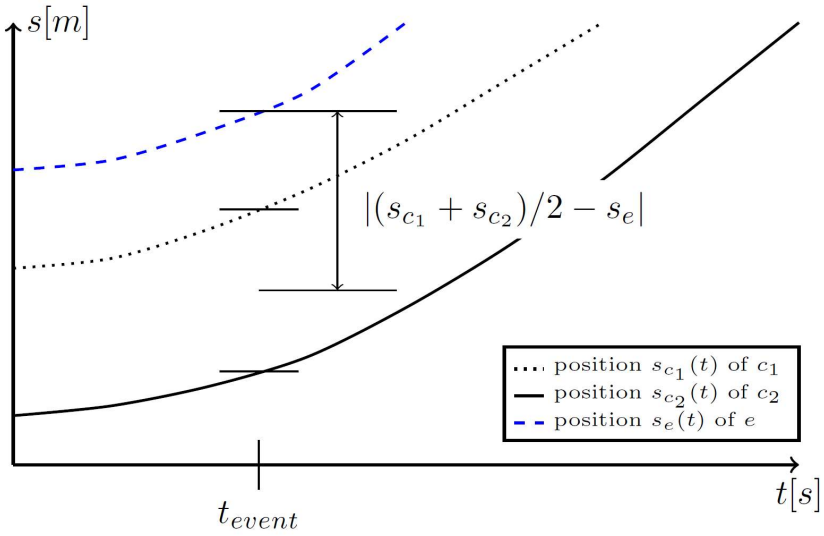


$$f_{idea,3} = \begin{cases} \left| \frac{z_{min}+z_{max}}{2} - value(t_1) \right|, & value(t_1) \text{ outside} \\ 0, & \text{otherwise} \end{cases}$$

Template 2a – Location

“The ego car should be in between the other cars.”

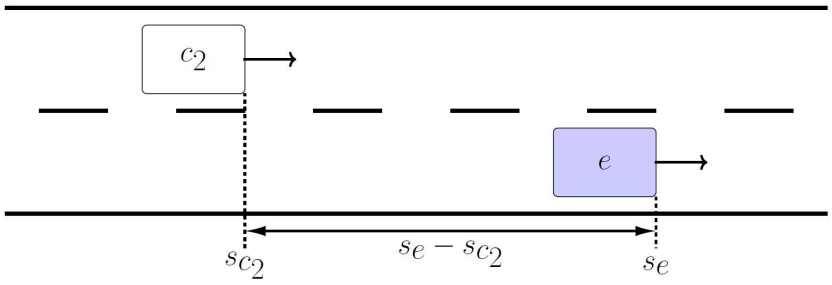


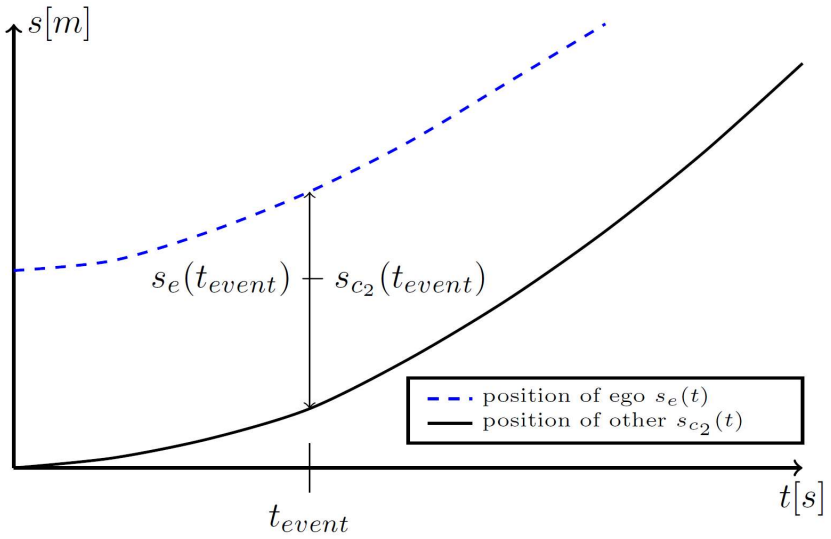


$$f_{template,2a} = \begin{cases} \left| \frac{s_{c_1}(t_{event}) + s_{c_2}(t_{event})}{2} - s_e(t_{event}) \right|, & e \text{ not in between } c_1 \text{ and } c_2 \\ 0, & \text{otherwise} \end{cases}$$

Template 2b - Location

“Other car should be in front of the ego car.”

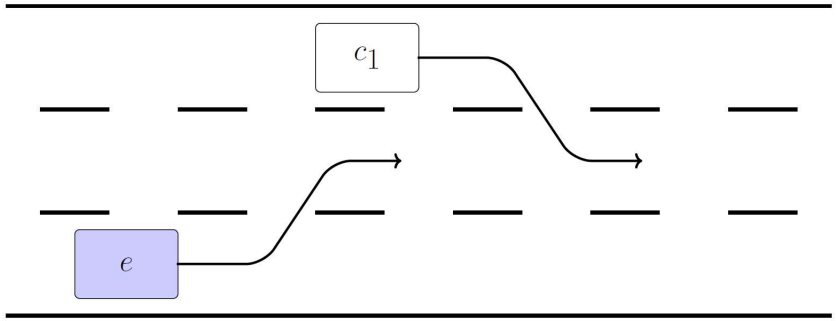


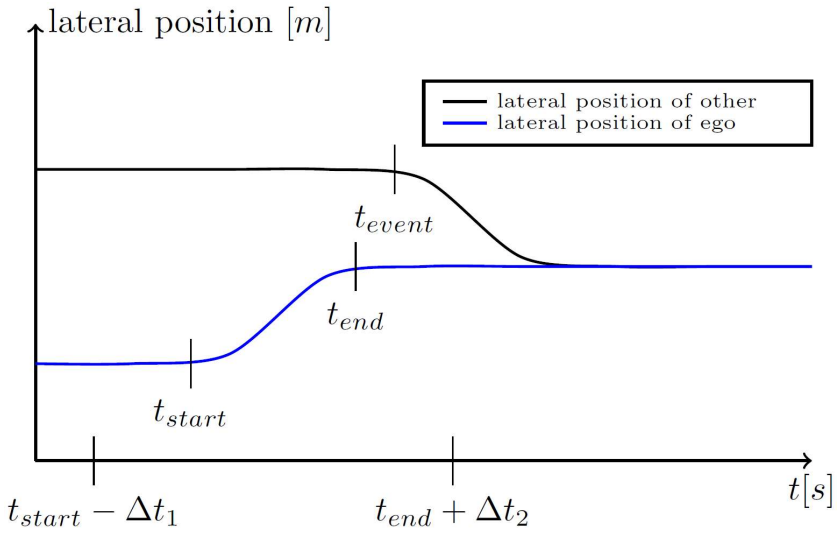


$$f_{template,2b} = \begin{cases} s_{c_2}(t_{event}) - s_e(t_{event}), & s_e(t_{event}) < s_{c_2}(t_{event}) \\ 0, & \text{otherwise} \end{cases}$$

Template 3 - Timing

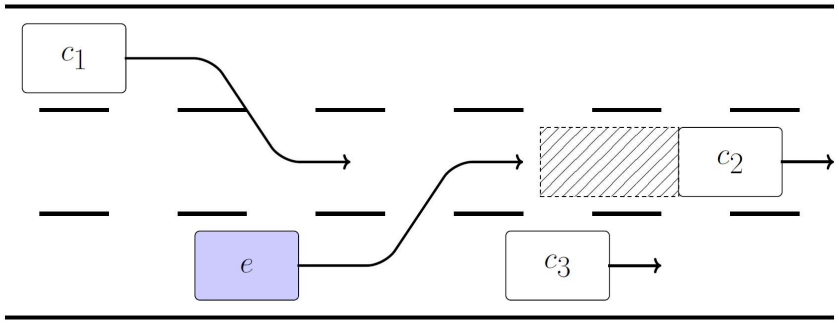
“Both cars should change onto the middle lane simultaneously.”





$$f_{template,3} = \begin{cases} \left| \frac{t_{start} - \Delta t_1 + t_{end} + \Delta t_2}{2} - t_{event} \right|, & t_{event} \text{ not in between bounds} \\ 0, & \text{otherwise} \end{cases}$$

Technical Solution for Combining the Templates



- lane change of e behind c_2
- simultaneous lane change of c_1
- lane change of c_1 behind e
- search for envelope violation

In case of single-objective optimizer:

$$f = \begin{cases} \alpha, & \text{lane change of } e \text{ not behind } c_2 \\ \begin{cases} \beta, & \text{lane change of } c_1 \text{ not simultaneously} \\ \begin{cases} \gamma, & \text{lane change of } c_1 \text{ not behind } e \\ \delta, & \text{search for envelop violation} \end{cases} \end{cases} \end{cases}$$

In case of multi-objective optimizer:

$$[a, \quad b, \quad c, \quad d]$$

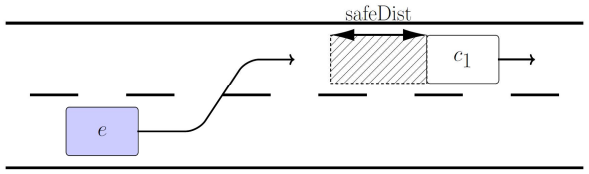
$$f_1 = \begin{cases} \alpha, & \text{lane change of } e \text{ not behind } c_2 \\ 0, & \text{otherwise} \end{cases}$$

$$f_2 = \begin{cases} \beta, & \text{lane change of } c_1 \text{ not simultaneously} \\ 0, & \text{otherwise} \end{cases}$$

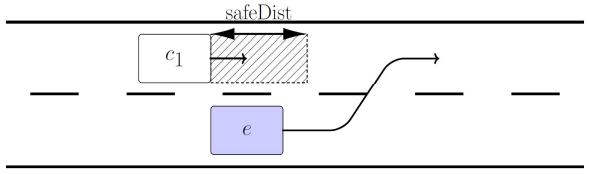
$$f_3 = \begin{cases} \gamma, & \text{lane change of } c_1 \text{ not behind } e \\ 0, & \text{otherwise} \end{cases}$$

$$f_4 = \delta \qquad \text{search for envelop violation}$$

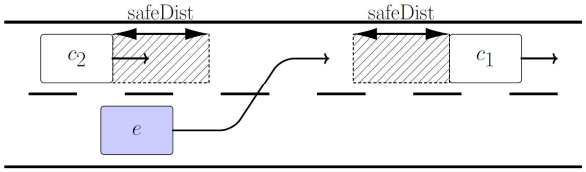
Covering More Lane Changes



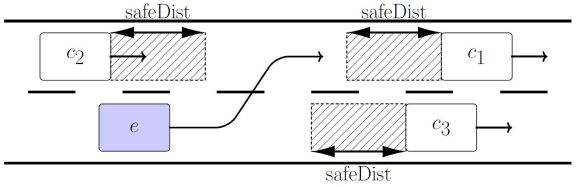
$$f = \begin{cases} 1000; & \text{no lane change happens} \\ s_e(t_{lc}) - s_{c1}(t_{lc}); & s_e(t_{lc}) >= s_{c1}(t_{lc}) \\ s_{c1}(t_{lc}) - s_e(t_{lc}) - safeDist; & s_e(t_{lc}) < s_{c1}(t_{lc}) \end{cases}$$



$$f = \begin{cases} 1000; & \text{no lane change happens} \\ s_{c1}(t_{lc}) - s_e(t_{lc}); & s_{c1}(t_{lc}) >= s_e(t_{lc}) \\ s_e(t_{lc}) - s_{c1}(t_{lc}) - safeDist; & s_{c1}(t_{lc}) < s_e(t_{lc}) \end{cases}$$



$$f = \begin{cases} 1000; & \text{no lange change happens} \\ \left| s_e(t_{lc}) - \frac{s_{c1}(t_{lc}) - s_{c2}(t_{lc})}{2} \right|, & \text{not between} \\ min\{|s_{ci}(t_{lc}) - s_e(t_{lc})| - safeDist_i\}, & otherwise \end{cases}$$



$$f = \begin{cases} 1000; & \text{no lange change happens} \\ \left| s_e(t_{lc}) - \frac{s_{c1}(t_{lc}) - s_{c2}(t_{lc})}{2} \right|, & \text{not between} \\ min\{|s_{ci}(t_{lc}) - s_e(t_{lc})| - safeDist_i\}, & otherwise \end{cases}$$

Signal Temporal Logic rather than Fitness Functions

Limits of Fitness Functions

- Comparing timed events requires stacking of multiple fitness function templates
- Only specific timepoints can be evaluated, not a signal/interval as a whole
- Harder to test for noncritical behavior, like efficiency or comfort
- Hard to encode actual traffic laws into fitness function templates
- Most traditional fitness functions are built for short and highly specific scenarios, not for evaluating long periods of autonomous driving

Stop Sign Example

How would you test if an AV is stopping correctly at stop signs?

StVO regarding stop signs[2]:

1. A person operating a vehicle must stop and give way.
2. A person operating a vehicle must not stop up to 10 meters in front of this sign if this would conceal the sign.
3. If there is no stop line (sign 294), they must stop at a place from which they have an unobstructed view of the other road.

[3]

STL is a formula that is used to test for properties or behaviors of time-series data

STL does this by incorporating normal, already known logical operators: **and, or, not**

with the specific time related operators:

- Eventually – F - ◇
- Always – G - □
- Until – U
- Implies - ⇒

STL Grammar

$= \mathrm{T} \mid \mu \geq$ (True, not) $\varphi_1 \wedge \varphi_2 \mid \varphi_1 \mathrm{U}_{[a,b]} \varphi$ (Until)

Additionally constructable: (predicate, and)

[4]

Or $\varphi_1 \vee \varphi_2 = \neg(\neg \varphi_1 \wedge \neg \varphi_1)$

Eventually $\Diamond_{[a,b]} \varphi = T \, U_{[a,b]} \varphi$

Always $\Box_{[a,b]} \varphi = \neg \Diamond_{[a,b]} \neg \varphi$

Implies $\varphi_1 \Rightarrow \varphi_2 = \neg \varphi_1 \vee \varphi_2$

STL Grammar II

Signal ***s*** at time **t** satisfies $\boldsymbol{\varphi}$

$(s, t) \vDash \varphi$

Predicate

The signal **s** at time **t** satisfies the predicate $\boldsymbol{\mu \geq 0}$ if $\boldsymbol{\mu}$ applied to the value of the signal at time **t** is greater than zero

$$(s, t) \vDash \mu \geq 0 \Leftrightarrow \mu(s(t)) > 0$$

Nonpositive values indicate violation

Logical Operators

Negation: $\neg \varphi$ (not φ)

$$(s,t) \vDash \neg\varphi \leftrightarrow (s,t) \nvDash \varphi$$ [1]

Conjunction: $\varphi_1 \wedge \varphi_2$ (φ_1 and φ_2)

$$(s,t) \vDash \varphi_1 \wedge \varphi_2 \leftrightarrow (s,t) \vDash \varphi_1 \; and \; (s,t) \vDash \varphi_2$$

Disjunction: $\varphi_1 \vee \varphi_2$ (φ_1 or φ_2)

$$(s,t) \vDash \varphi_1 \vee \varphi_2 \leftrightarrow (s,t) \vDash \varphi_1 \; or \; (s,t) \vDash \varphi_2$$ [1]

Eventually – F - ◇

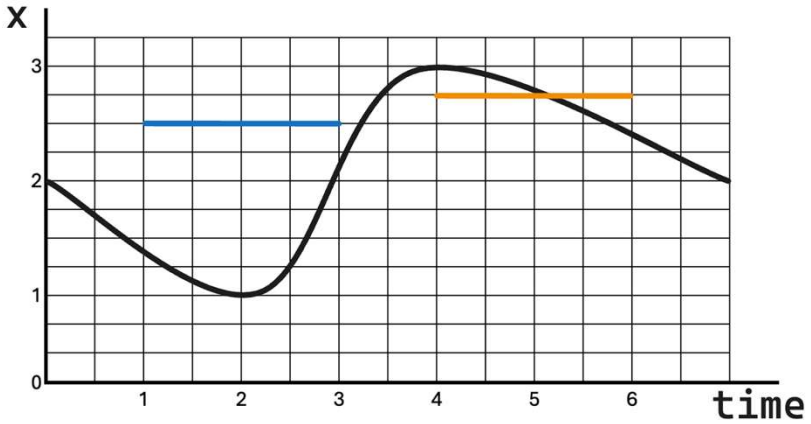
$$f = Eventually_{[a,b]}(\varphi) = F_{[a,b]}\varphi = \Diamond_{[a,b]}\varphi$$

$$(s,t) \vDash \Diamond_{[a,b]}\varphi \leftrightarrow \exists t' \in t + [a,b](s,t') \vDash \varphi \quad [1]$$

Formula φ holds at some time in the interval t + [a, b] = [t + a, t + b]

Example:

- $\Diamond_{[1,3]}$ (x > 2,5) at t=0 is False
- $\Diamond_{[1,3]}$ (x > 2,5) at t=3 is True
- $\Diamond_{[1,3]}$ (x < 2,5) at t=3 is also True



Always – G - □

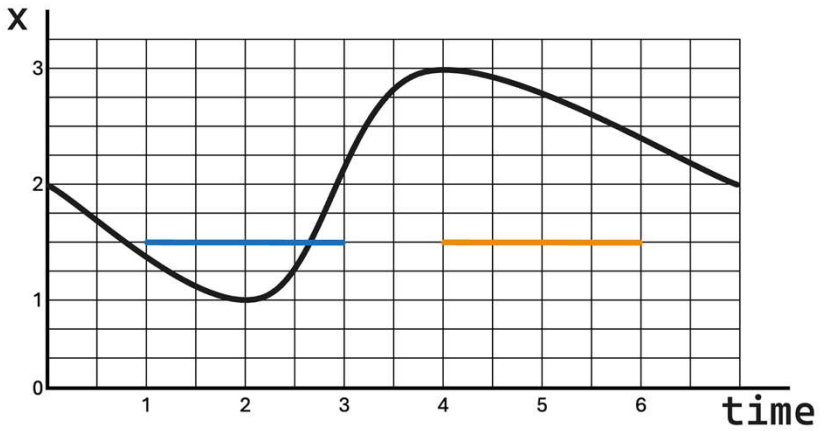
$$f = Always_{[a,b]}(\varphi) = G_{[a,b]}\varphi = \square_{[a,b]}\varphi$$

$$(s,t) \vDash \square_{[a,b]}\varphi \leftrightarrow \forall t' \in t + [a,b](s,t') \vDash \varphi \quad [1]$$

Formula φ always holds in the interval t + [a, b] = [t + a, t + b]

Example:

- $\square_{[1,3]}$ (x > 1,5) at t=0 is False
- $\square_{[1,3]}$ (x > 1,5) at t=3 is True



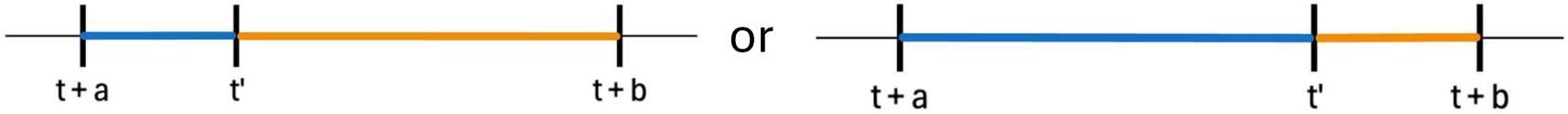
Until – U

$$f = \varphi_1 \; Until_{[a,b]}(\varphi_2) \; = \varphi_1 \; \mathsf{U}_{[a,b]} \varphi_2$$

$$(s,t) \vDash \varphi_1 \mathsf{U}_{[a,b]} \varphi_2 \; \leftrightarrow \; \exists t' \in \; [t+a, t+b](s,t') \vDash \varphi 2 \; and$$

$$\forall t'' \in [t,t'], (s,t'') \vDash \varphi_1 \quad [1]$$

Formula φ_1 holds until $t' \in \; [t+a, t+b]$. After t', φ_2 holds

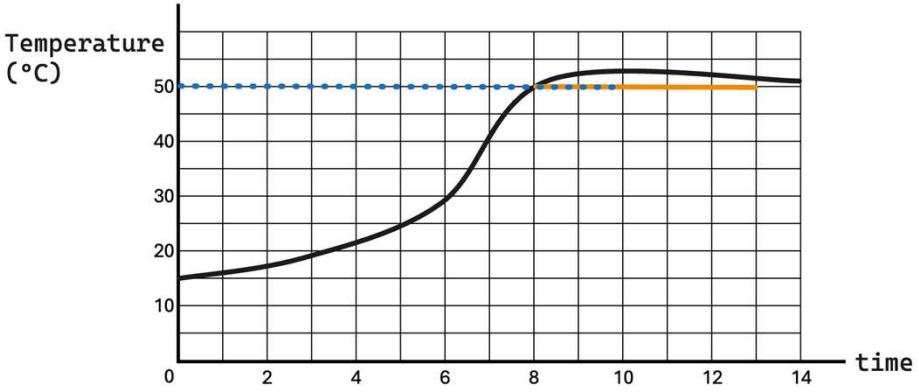


Combination

Consider a temperature control system where the temperature ***T(t)*** must:

- eventually rise above **50** degrees within the next **10** seconds
- then remain above **50** degrees for at least **5** seconds.

$\Diamond_{[0,10]}(\Box_{[0,5]}(\mathrm{T}(\mathrm{t}) > 50))$



Stop Sign Solution

1. A person operating a vehicle must stop and give way.
2. A person operating a vehicle must not stop up to 10 meters in front of this sign if this would conceal the sign.
3. If there is no stop line (sign 294), they must stop at a place from which they have an unobstructed view of the other road.

$$f_1 = (signAhead.type == stop) \Rightarrow \Diamond_{[0,2]} (\Box_{[0,3]} (speed == 0) \wedge stoplineAhead)$$

$$f_2 = (signAhead.type == stop \wedge signAhead.distance \leq 10 \wedge concealedByEgo) \Rightarrow speed > 0$$

$$f_3 = (signAhead.type == stop) \wedge \neg stoplineAhead.exists \Rightarrow$$
$$\Rightarrow \Diamond_{[0,2]} (\Box_{[0,3]} (speed == 0) \wedge intersectionAhead)$$

$$f_{final} = \Box(f_1 \wedge f_2 \wedge f_3)$$

Note: It is recommended to stop 3 seconds at stop signs

STL Example

Yellow traffic light and too little braking distance -> cross the intersection

$$f_1 = \square\Big((yellow \wedge d \geq 0 \wedge \neg canStopBeforeLine \wedge clear) \Rightarrow (v > \varepsilon_v)\, U_{[0,T_c]}\, JunctionCrossed\Big)$$

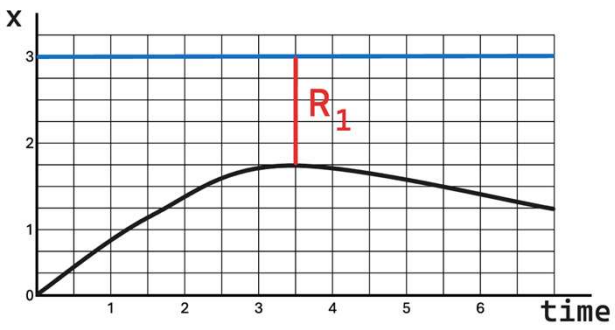
For reaction time t_r, decelaration a: $canStopBeforeLine \equiv d \geq v \cdot t_r + \frac{v^2}{2a}$

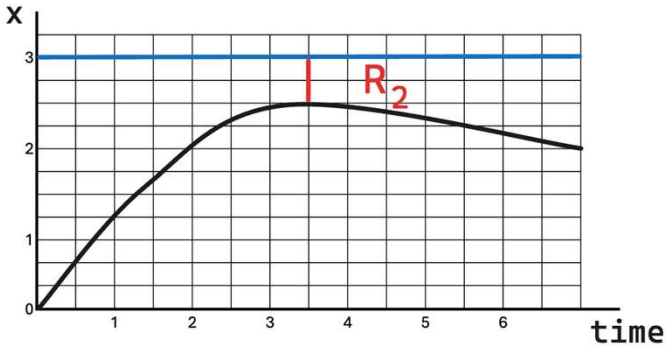
Robustness

So far, STL only tells us if a given signal violates a given formula

Optimizers need a fitness function, making the performance of two test-cases comparable -> Robustness value

Example: how close were the signals to speed limit **x**





$R_1 > R_2$ → R_2 will be used/adapted to find more critical tests

Calculating Robustness value ρ

Predicate

$$\rho(s, t, \mu \geq 0) = \mu(s(t))$$

Not

$$\rho(s, t, \neg\varphi) = -\rho(s, t, \varphi)$$

And

$$\rho(s, t, \varphi_1 \wedge \varphi_2) = min(\rho(s, t, \varphi_1), \rho(s, t, \varphi_2))$$

Or

$$\rho(s, t, \varphi_1 \vee \varphi_2) = \max(\rho(s, t, \varphi_1), \rho(s, t, \varphi_2))$$

Calculating Robustness value ρ

Eventually

$$\rho\left(s, t, \Diamond_{[a,b]}\varphi\right) = max_{\,t' \in [t+a,\, t+b]}\, \rho(s, t', \varphi)$$

Always

$$\rho\left(s, t, \Box_{[a,b]}\varphi\right) = min_{\,t' \in [t+a,\, t+b]}\, \rho(s, t', \varphi)$$

Until

$$\rho(s, t, \varphi_1\, U_{[a,b]}\varphi_2) = max_{\,t' \in [t+a,\, t+b]}\, (\min(\rho(s, t', \varphi_2), min_{\,t'' \in [0,\, t']}\, \rho(s, t'', \varphi_2)))$$

Implies

$$\rho(s, t, \varphi_1 \Rightarrow \varphi_2) = \max(-\rho(s, t, \varphi_1), \rho(s, t, \varphi_2))$$

STL Example

Yellow traffic light and too little braking distance -> cross the intersection

$$f_1 = \square\Big((yellow \wedge d \geq 0 \wedge \neg canStopBeforeLine \wedge clear) \Rightarrow (v > \varepsilon_v)\, U_{[0,T_c]}\, JunctionCrossed\Big)$$

For reaction time t_r, decelaration a: $canStopBeforeLine \equiv d \geq v \cdot t_r + \frac{v^2}{2a}$

$$\rho(f_1, x, t) = \inf_{t' \geq t} \max\Big(-\rho(\alpha, x, t'),\ \rho(\beta, x, t')\Big)$$

with

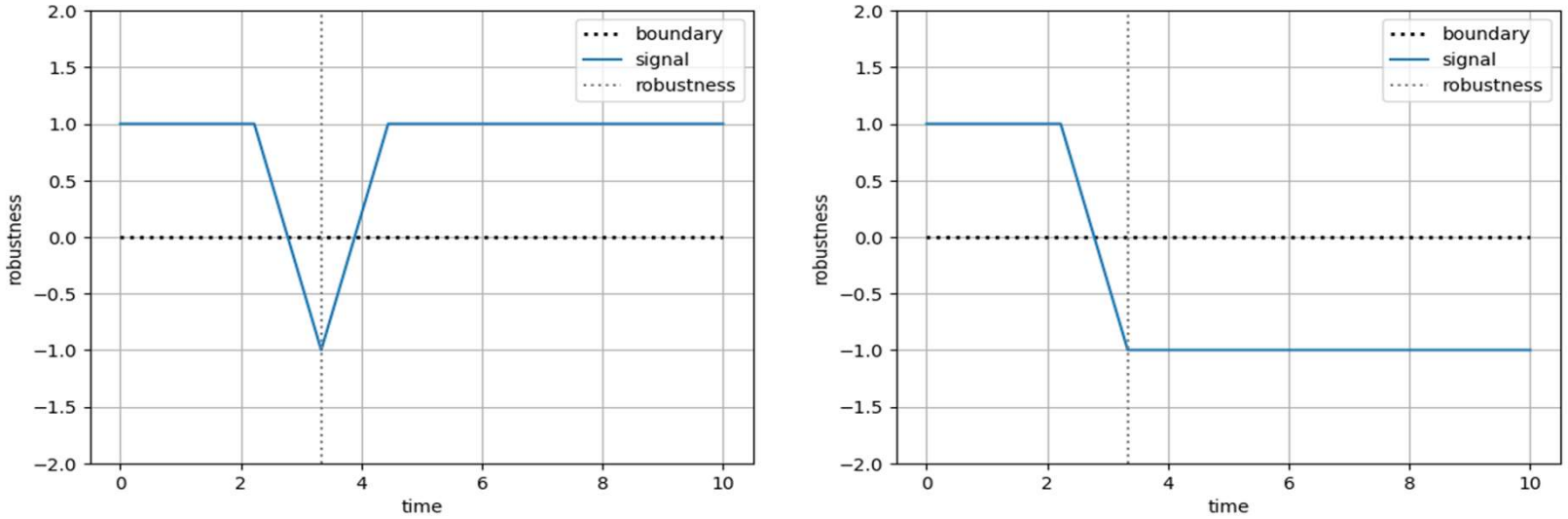
$$\rho(\alpha, x, t') = \min\Big(\rho(yellow),\ d(t'),\ vt_r + \tfrac{v^2}{2a} - d(t'),\ \rho(clear)\Big)$$

and the Until:

$$\rho(\beta, x, t') = \sup_{t'' \in t' + [0,T_c]} \min\Big(\rho(JC, t''),\ \inf_{t''' \in [t', t'']} \big(v(t''') - \varepsilon_v\big)\Big)$$

Problems of Standard STL

Pointwise Masking



➔ Both signals have same robustness; duration of violation ignored

Out of Bounds Information/Robustness

Input Signals masking Output Signals

Pointwise Masking Solution

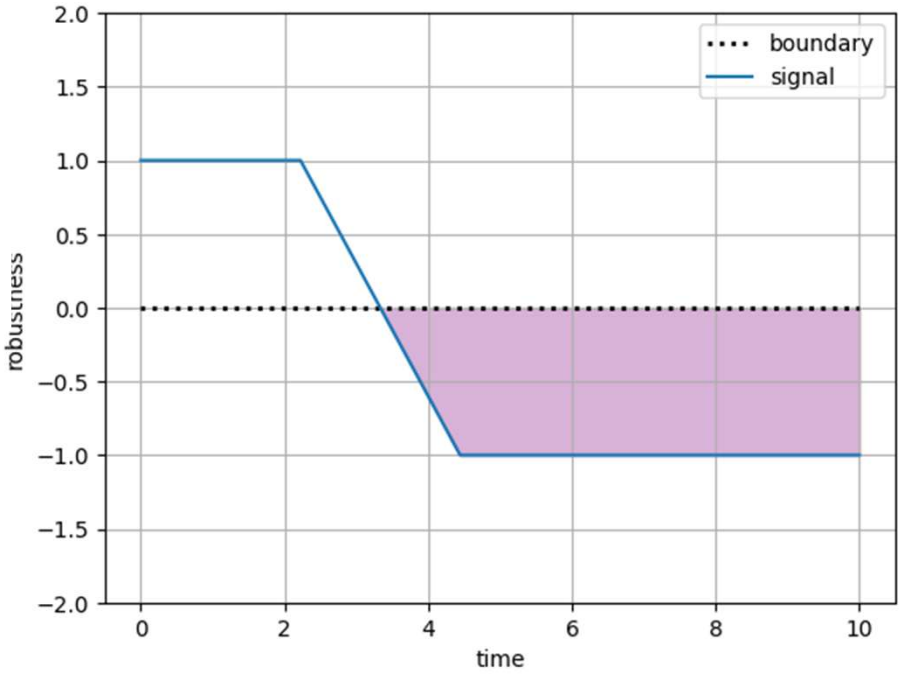
Generalized Mean Robustness

Duration Robustness

$$\rho(\mathbf{H^D}_I\varphi, \boldsymbol{\omega}, t) = \begin{cases} \inf(\mathcal{X}) & \text{if } \mathcal{X}_{<0} = \emptyset, \\ -\frac{|\mathcal{X}_{<0}|}{|\mathcal{T}'|} & \text{otherwise.} \end{cases}$$

Duration-Severity Robustness:

$$\rho(\mathbf{H^{DS}}_I\varphi, \boldsymbol{\omega}, t) = \begin{cases} \inf(\mathcal{X}) & \text{if } \mathcal{X}_{<0} = \emptyset, \\ \frac{\sum_{x \in \mathcal{X}_{<0}} x}{|\mathcal{T}'|} & \text{otherwise.} \end{cases}$$

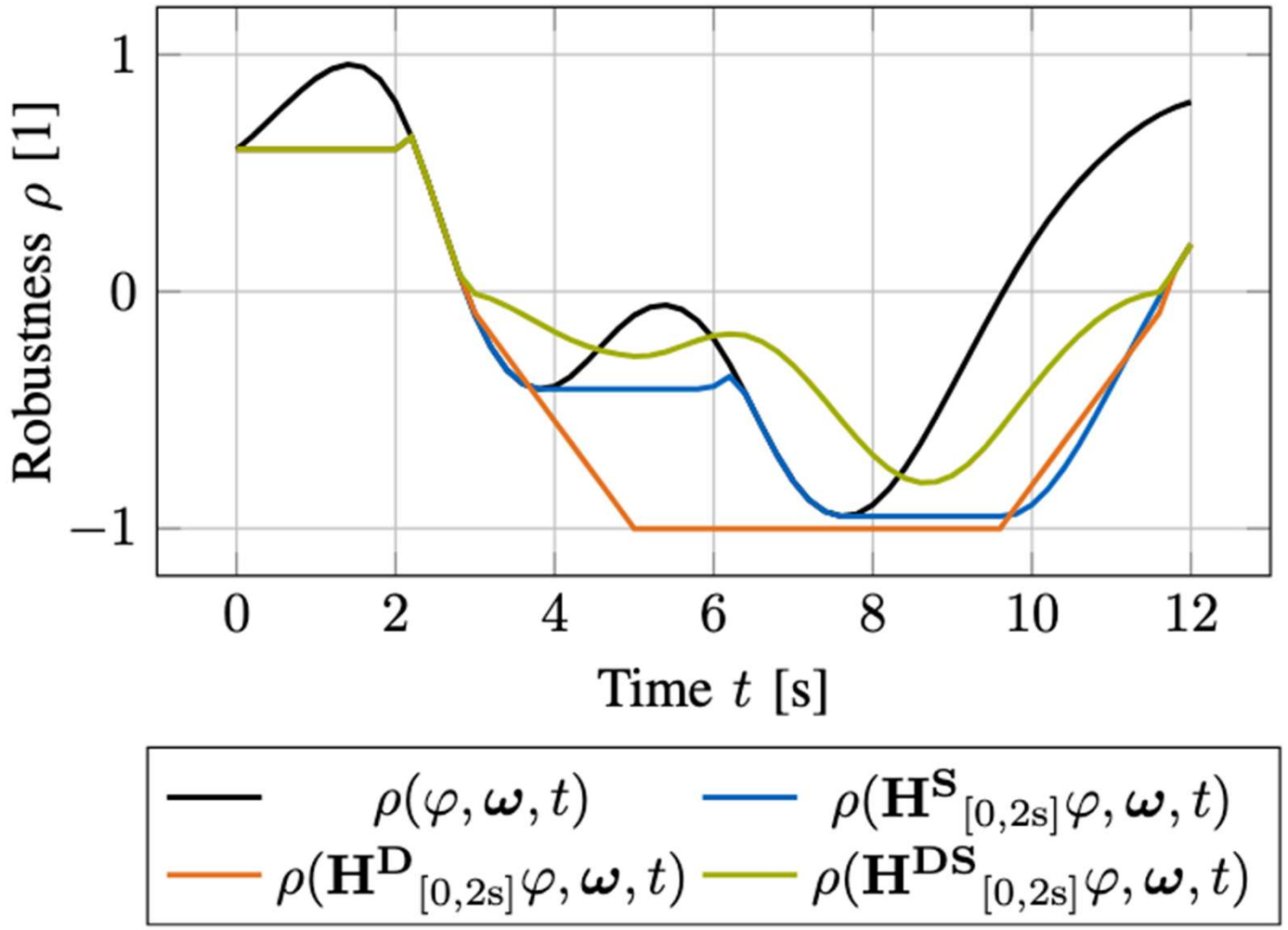


➔ Combine severity & duration of the violation in the robustness calculation

N. Mehdipour, C.-I. Vasile, and C. Belta, “Generalized mean robustness for signal temporal logic,” IEEE Transactions on Automatic Control, vol. 70, no. 3, pp. 1949–1956, 2025.

F. Finkeldei, M. Wolf, J. -N. Weghorn, A. Pretschner and M. Althoff, "Enhanced Traffic Rule Monitoring Using Model-Predictive and Duration-Aware Robustness," 2025 IEEE 28th International Conference on Intelligent Transportation Systems (ITSC), Gold Coast, Australia, 2025, pp. 1322-1329, doi: 10.1109/ITSC60802.2025.11423597.

Different Robustness Measures compared



Summary

Signal Temporal Logic offers a formal way to specify spatio-temporal relationships of continuous signals

Calculating the robustness offers us a way to use STL-formulas as fitness functions

But there is not just one notion of robustness but many: “is violating once as bad as over an extended period of time?”

Applications:

- Scenario-based Testing
- Evaluation of large amounts of driving data
- Motion-Planning , e.g.: □(◇(reach a gas station))

Lots of different STL-extension to tackle specific problems

So far …

1. We know we cannot re-use tests and need to generate tests from scenario types for each new version of a subsystem
2. Given a set of scenario types, we know how to derive „good“ scenario instances (= test cases) for each:
   1. Translate essence of scenario type into optimization problem (not trivial)
   2. Then heuristically search for violations of the safety envelope

      Violation found? Good: problem in car revealed.

      No violation found? Also good: tests are good when targeting *potential* defects.

But is our list of scenario types complete?

Completeness

… of scenario types w.r.t. reality

… of tests w.r.t. one scenario type

… of scenario types w.r.t. recorded data (purely statistical argument)

[Hauer, Schmidt, Holzmüller, P: Did We Test All Scenarios for Automated and Autonomous Driving Systems? ITSC 2019]

… of clustered data w.r.t. expert-defined scenario types

… of scenario types w.r.t. granularity

… of data w.r.t. reality

Use clustering to transform recorded drives into scenario types, and then compare these to hand-crafted scenario types

This is where recorded drives help.

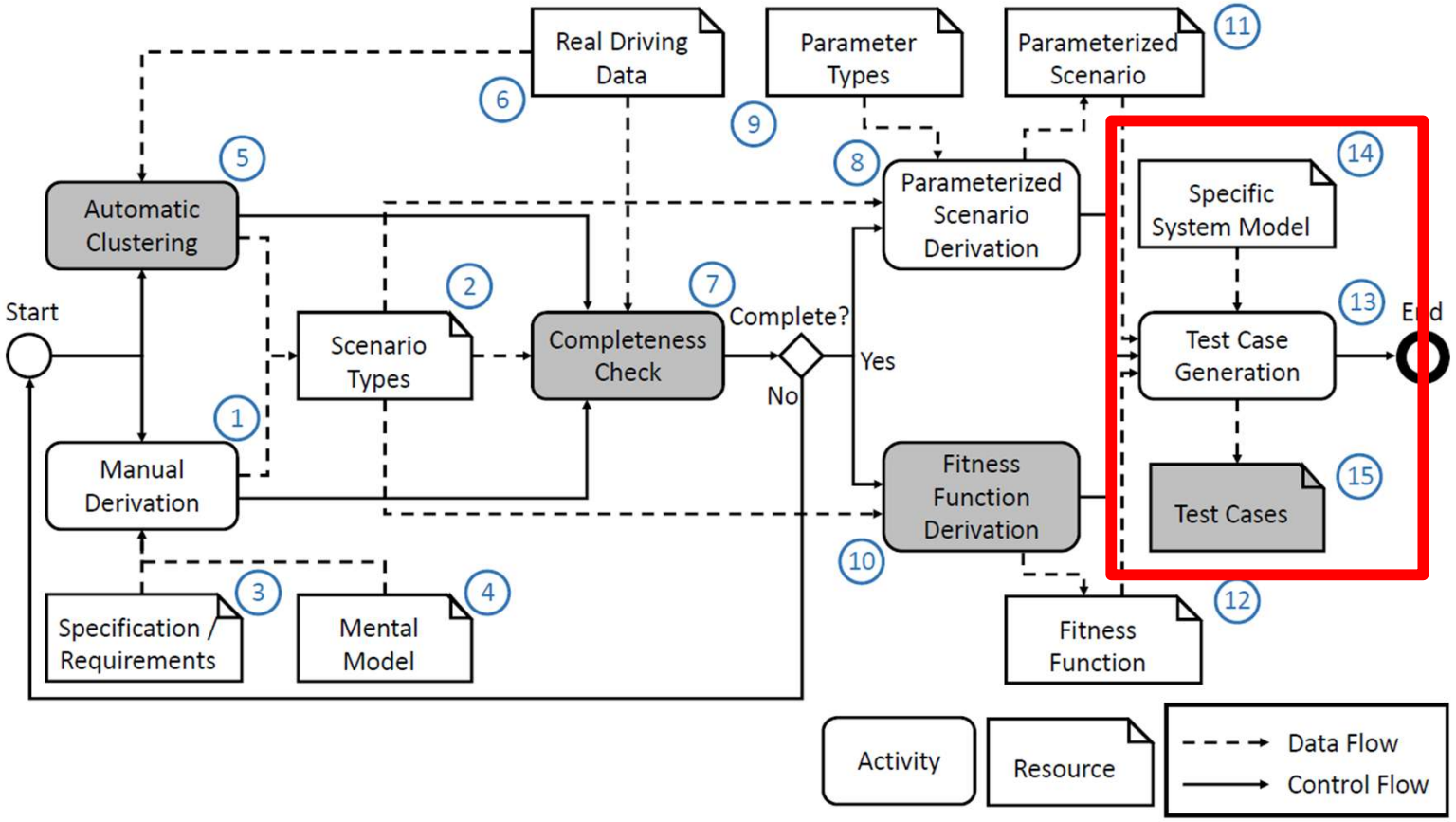
Summary and Discussion

- Virtual testing mandatory

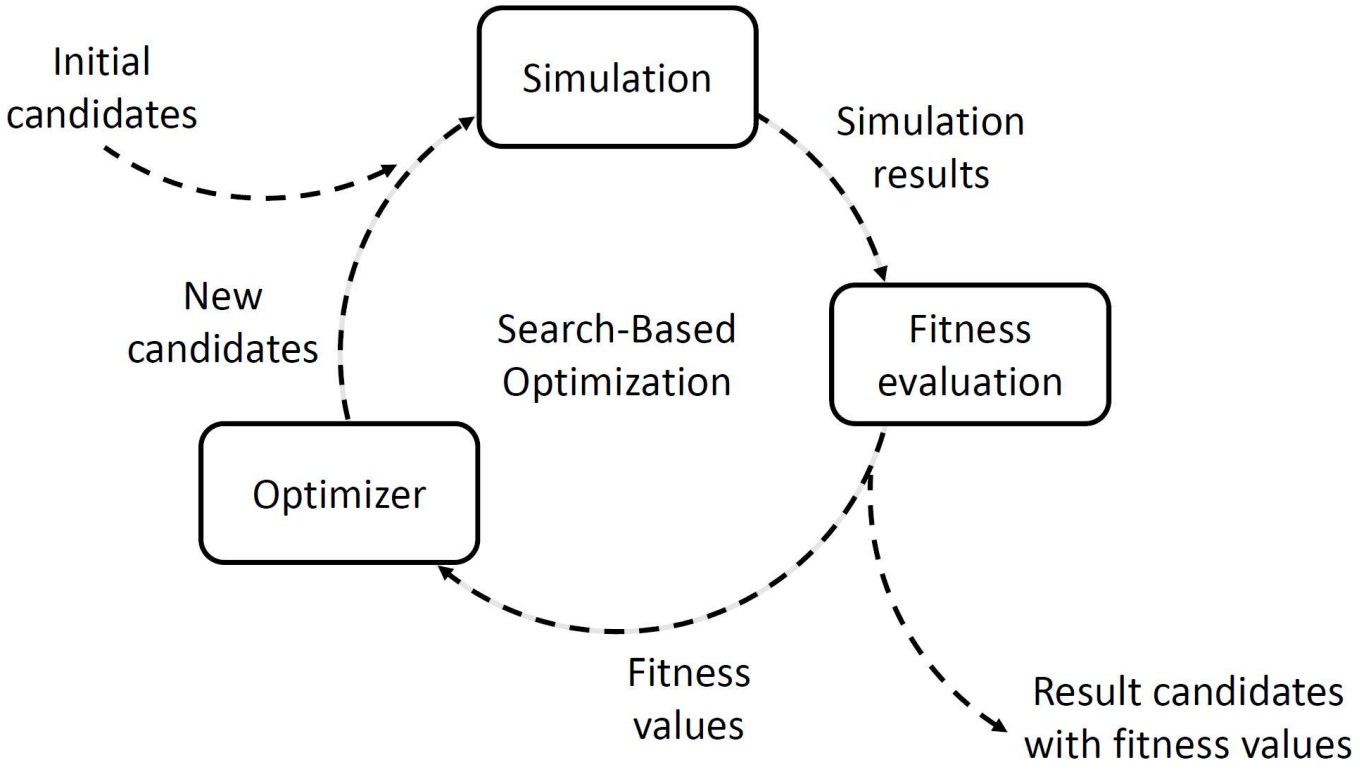
- Scenarios define spatial and temporal constraints;
  encode in fitness function via templates
- Quality of tests encoded as optimization goal:
  directly or via robustness in STL
  In general no re-use/regression testing, hence re-generation.
  Expensive; but can reduce effort via clever seeding
- No global optimum:
  different algorithm combinations lead to/don‘t lead to crash
- Completeness of scenario types

- Didn‘t discuss sim2real gap
- Didn‘t discusss scalability issues

Test Case Generation



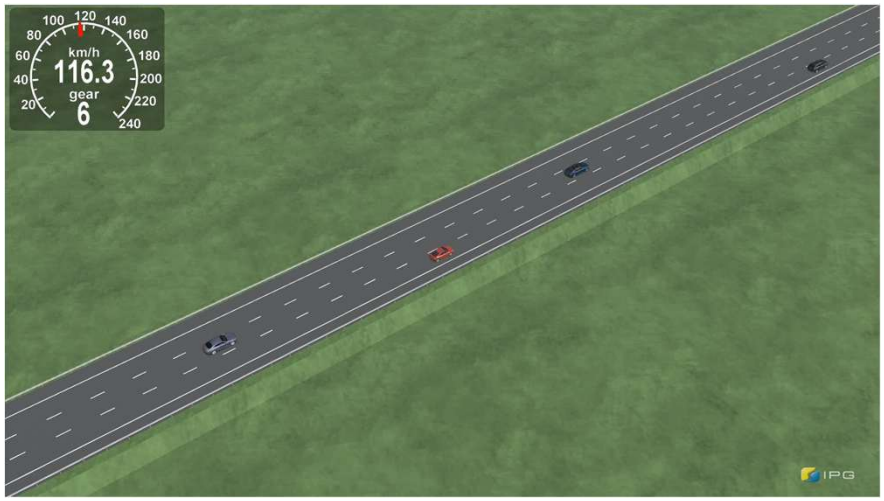
Search-Based Test Selection



Examples

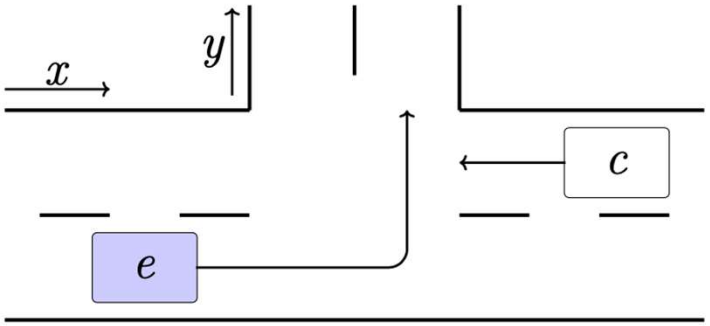

km/h
83.6
gear
4
IPG


km/h
114.5
gear
5
IPG


km/h
116.3
gear
6
IPG


km/h
103.5
gear
6
IPG

Generalization: Intersections



Parameter	Lower Bound	Upper Bound
Starting velocity $v_{e,start}$ [m/s]	5.5	13.9
Starting time $t_{c,start}$ [s]	0	20
Participant velocity v_c [m/s]	6.9	8.3

- Left turn of e before c
- AND search for envelope violation in longitudinal and lateral direction

However, avoid potential pitfalls

- At least one search problem for each scenario type (strong dependence on scenario type granularity)
- One separate search problem for each safety criterion (e.g. simultaneous search for longitudinal and lateral envelope violation not useful!)

[Kolb, Hauer, P: Fitness Function Templates for Testing Automated and Autonomous Driving Systems in Intersection Scenarios. ITSC 2021]

No Global Optima with Heuristic Search

Different optimizing algorithms yield different results – as strong as „crash“ vs. „no crash“.

Example: testing the PX 4 autopilot for drones w.r.t. collisions with different combinations of optimizers:

	NSGAII	PSO	BO	NPN	NPB	NBN	NBP	PNP	PNB	PBN	PBP	BNP	BNB	BPN	BPB
Performance	Base	-2%	-21%	+10%	-12%	+12%	+5%	+20%	+5%	+16%	+9%	+11%	-5%	+8%	-11%
P-value	Base	0.20	**0.00**	0.17	0.34	0.40	0.48	**0.01**	0.32	0.11	0.17	0.22	0.26	0.33	**0.04**
A_{12}	Base	0.46	**0.26**	0.55	0.49	0.52	0.51	**0.57**	0.53	**0.56**	0.54	0.51	0.44	0.50	**0.39**

Schmidt, Pretschner: StellaUAV: A Tool for Testing the Safe Behavior of UAVs with Scenario-Based Testing. Proc. ISSRE, pp. 37-48, 2022.

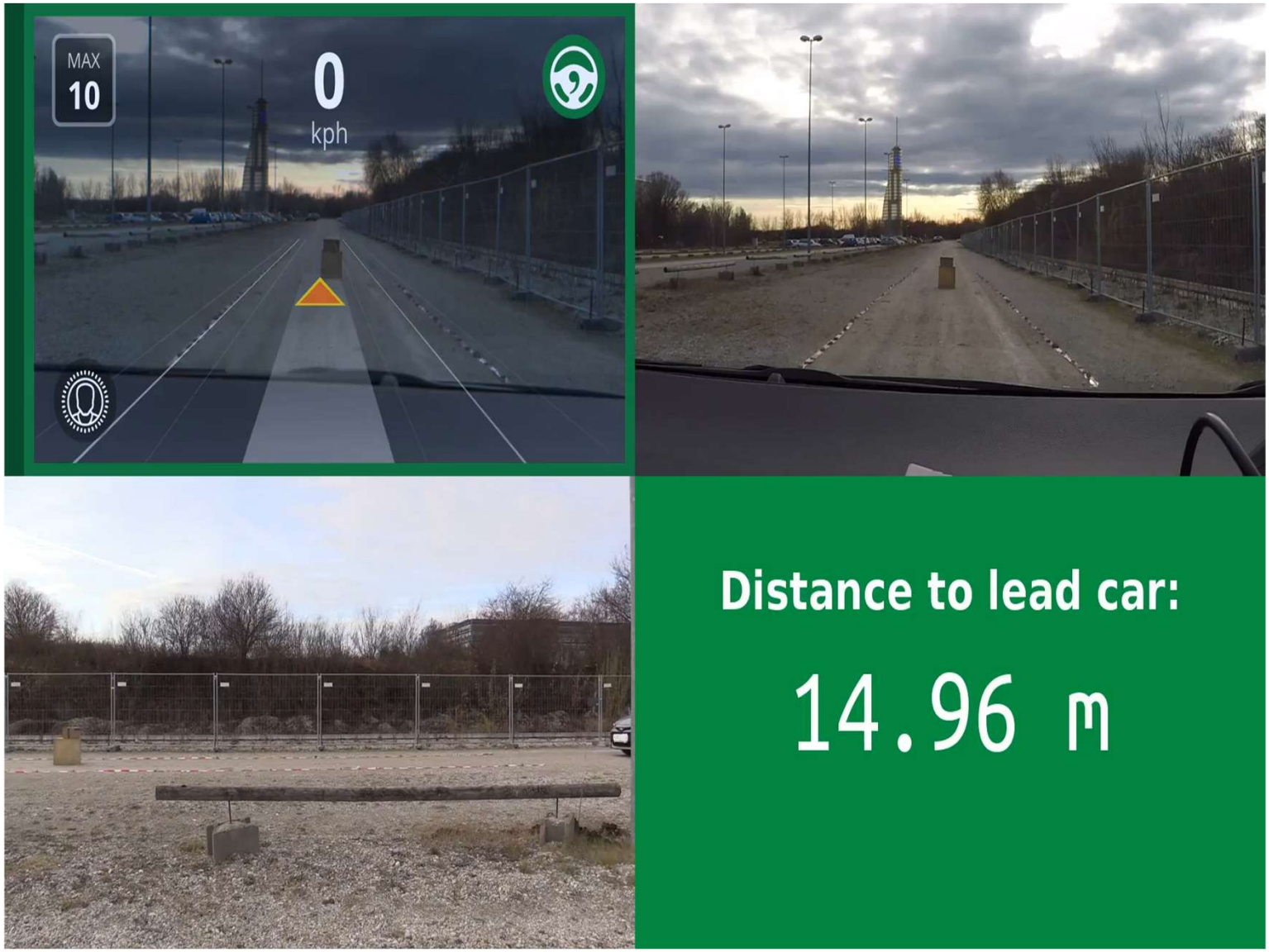
Result: Can‘t trust one algorithm, but there also is no best combination.
[Way out?]

Application: Generate attacks to overcome IDS (SiL, real car)

Warm-Up: Inject wrong Steering-Wheel Angle

[Sensor inputs are lane marking and steering wheel angle: ADAS tries to align, or counteract. Faking the steering wheel angle makes ADAS react even though it should not.]

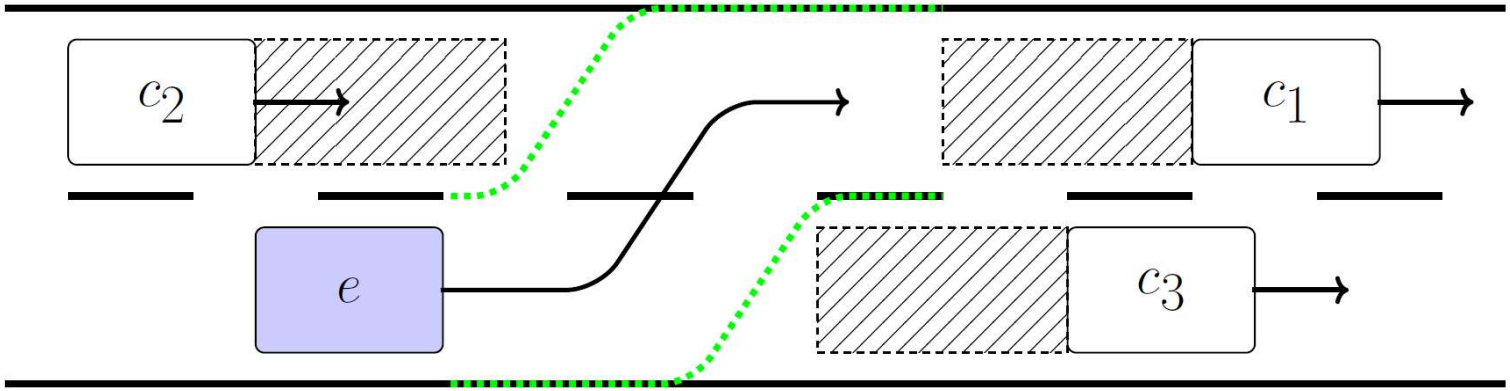
Vehicle-In-The-Loop



Attack found using test case generation: Pretend steering wheel was not moved

[Sensor inputs are lane marking and steering wheel angle: ADAS tries to align.
Cars never drive perfectly straight: ADAS needs to correct continuously. If transmitted steering wheel angle is constant even though the wheel actually moves, the ADAS „over“corrects, leading to oscillations.
Attack found by a machine, not a human.]

Revisiting Re-Use: Trajectory Planner



1. Predict positions of other cars

2. Determine safety corridor / safe operation envelope
   using $safeDistEstimation = \max\{\epsilon\ ,\ \tau\ * v_{other}\}$

3. Compute optimal trajectory
   using $\sum_i^N \theta\left(v_{planned,i}\ - v_{desired}\right) + \kappa * a_i^2$

Approach developed by VolvoCar: Nilsson, Lane Change Maneuvers for Automated Vehicles, 2017

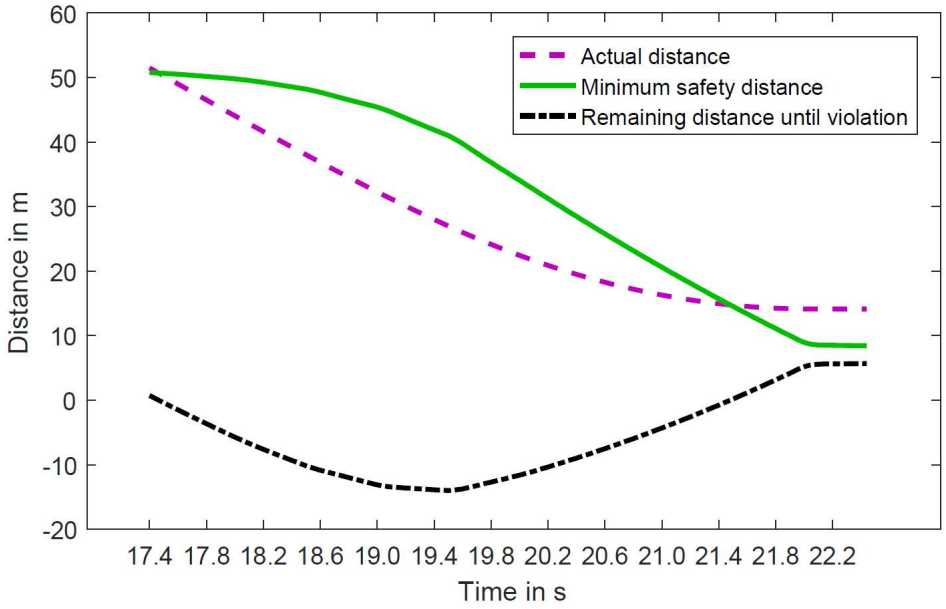
Three experiments

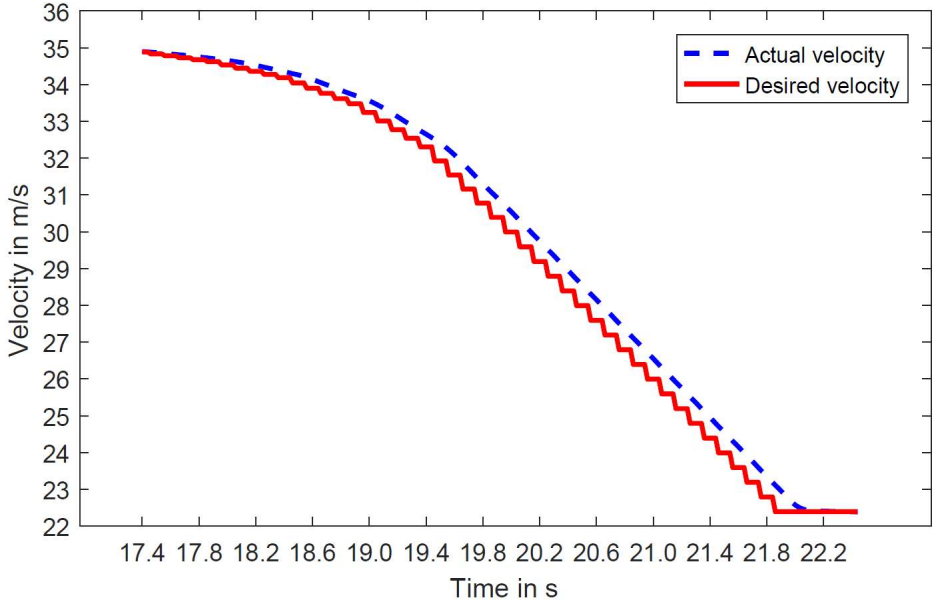
Try to collide with vehicle c1 for

System A: baseline
System B: like A but larger required safety distance in trajectory planner
System C: like B but proportional gain of PID controller is reduced

This is plain old limit testing, hence defect-based, hence „good“!

Results: Experiment A



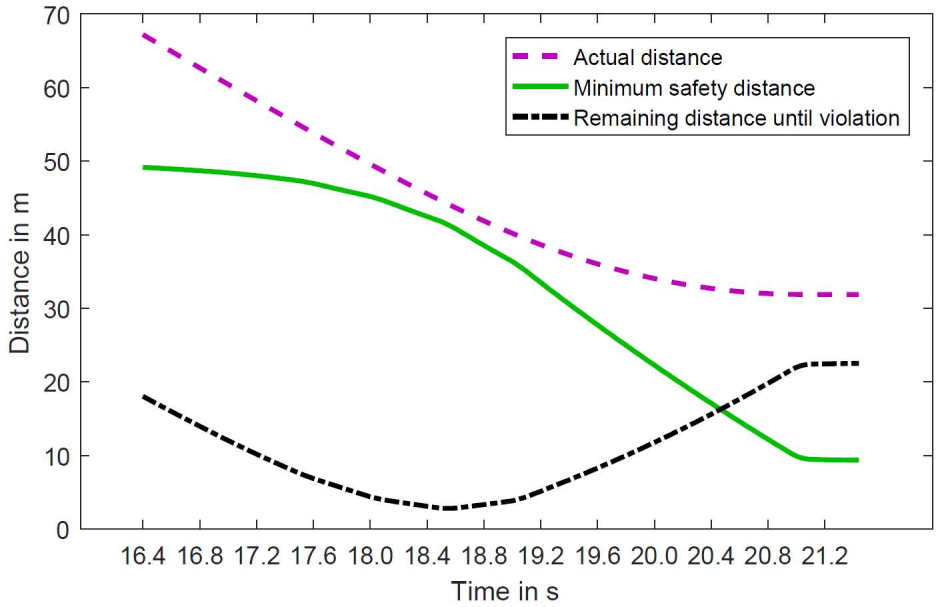


Safety distance violation: Trajectory planner does not consider sufficient distance

Results: Experiment B

Increase minimum distance within the trajectory planner

Rerun experiment

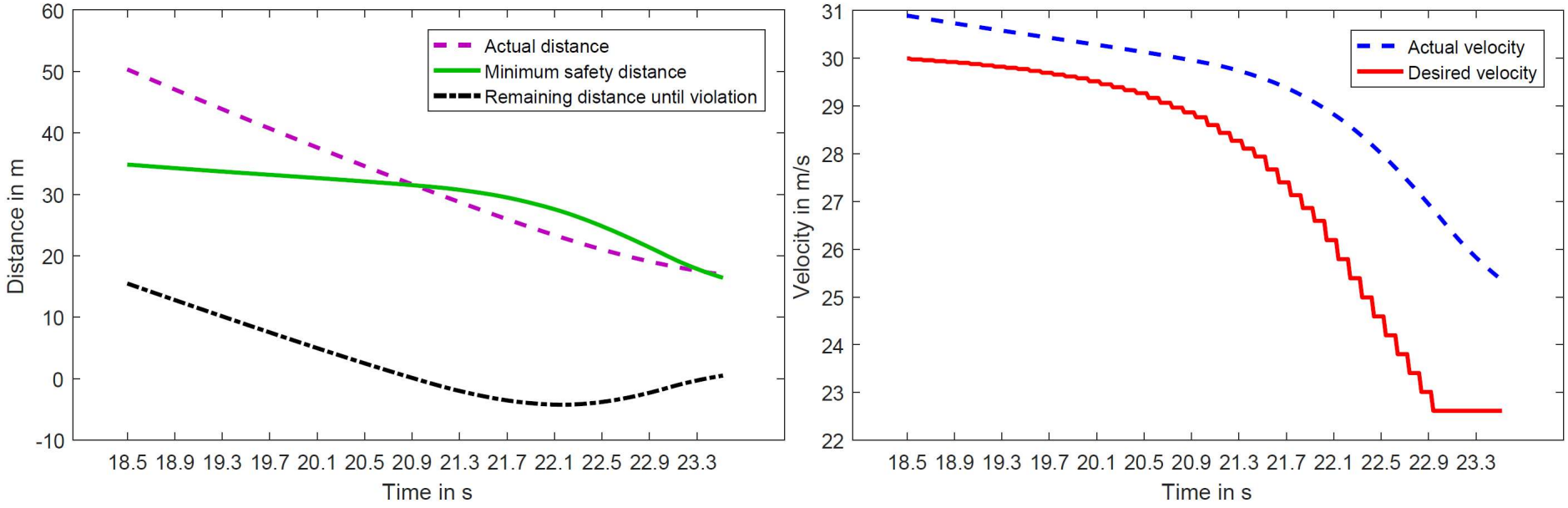


No safety distance violation: System is assumed to be correct

Results: Experiment C

Decrease proportional gain of longitudinal tracking: Increase control smoothness

Rerun experiment



Safety distance violation: Tracking too slow

Re-using Scenarios?

Each system (A, B, C) leads to generation of a “worst case” test case – including parameters of the other cars. Let’s now re-use these generated scenarios (which could as well be recorded drives!) as tests for the respective other systems.

	System A	System B	System C
Scenario A	-14.001	5.604	-3.065
Scenario B	2.595	2.812	-3.037
Scenario C	20.153	20.484	-4.238

Worst case scenarios are system specific!

=> Reusing concrete scenarios is (generally) not a good idea

Hauer, Pretschner, Holzmüller: Re-Using Concrete Test Scenarios Generally Is a Bad Idea. IV 2020: 1305-1310

Ouch!

Recap: A good test for one version is not necessarily good for the next version – could as well be picked at random.
Yet, cost of full re-generation in a CI setting prohibitive.

Intuition: Best test will be different, but not by a lot!
Way out: Use best test case for last version as initial seed for heuristic search.
Yields better tests faster (reduction by ~70% when compared with random seed re-generation)

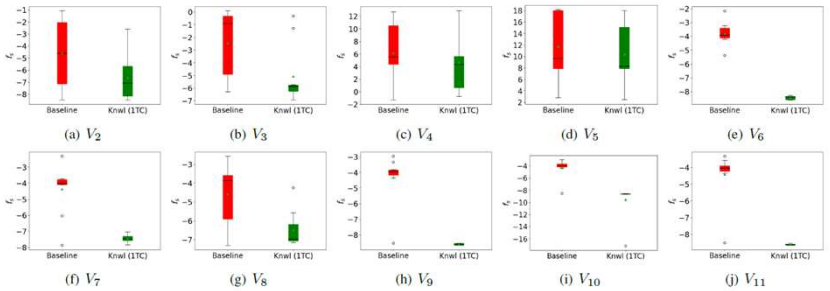


FIGURE 6. Box-plots on the f_δ values of the best test cases found aggregated for the baseline test case re-generation runs (red) and the knowledge-based test case re-generation runs (knwl) (green). Lower f_δ values correspond to better test cases.

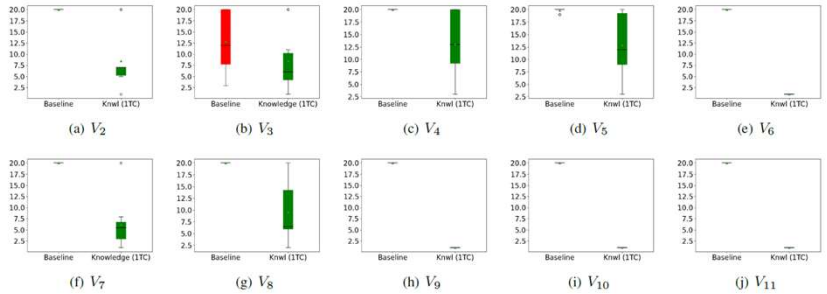


FIGURE 7. Box-plots on the population generation information for the baseline test case re-generation runs (red) and the knowledge-based test case re-generation runs (knwl)(green) for the individual adaptations.

Kolb, Pretschner: Knowledge is Power in Search-based Test Case Generation for Autonomous Vehicles. Submitted, 2024